FEU Institute of Technology
COLLEGE OF ENGINEERING • COLLEGE OF COMPUTER STUDIES

Computer System and Architecture

Machine Problem

FINAL PROJECT

Charles Gabriel Marcelo	Doc. Hadji Tejuco
Name of Student	Name of Professor
November 6, 2026	November 12, 2026
Data Performed	Date Submitted

List the steps that you perform to come up with the program.

1. **Selected the application concept.**
   Designed Byte Bites as a food-ordering application where customers can select multiple food items, review their orders, pay, and view saved receipts.

2. **Defined the food items and prices.**
   Added cheeseburgers at ₱80, chicken rice at ₱120, French fries at ₱35, and iced tea at ₱25.

3. **Designed the menu and ASCII artwork.**
   Created a store banner and illustrations of the four food items. Added menu options for displaying food, adding items, viewing the order, checking out, reading receipts, cancelling, and exiting.

4. **Organized the program data.**
   Stored messages, food names, and prices in the .data section. Reserved space in .bss for the cart, total, payment, change, and input and file buffers.

5. **Implemented keyboard input and validation.**
   Created the read_number procedure to process numeric input. Added checks for valid menu choices, food IDs, quantities, and payment amounts. Limited each food item to 99 units per order.

6. **Developed the ordering functionality.**
   Implemented adding food quantities to the cart, calculating item costs, updating the total, displaying the current order, and clearing cancelled orders.

7. **Implemented checkout and receipt generation.**
   Added checks for an empty order and insufficient payment. Calculated change, requested payment confirmation, and used build_receipt to prepare a receipt containing the purchased items, total, cash, and change.

8. **Created reusable macros and procedures.**
   Used the PRINT macro to simplify message display. Organized reusable operations into procedures such as print_z, write_all, append_uint, clear_cart, and build_receipt.

9. **Added file writing and reading.**
   Implemented save_receipt to append confirmed purchases to receipts.txt. Added read_history to read and display receipts from previous transactions.

10. **Handled errors and completed the transaction flow.**
    Added messages for invalid input, insufficient payment, missing receipt files, and file-operation failures. The cart is cleared after a successfully saved purchase and retained if saving fails.

Program Description

Byte Bites is a food shop simulation game developed using 32-bit NASM assembly language for Linux. The player manages a small food business by serving customers, purchasing ingredients, upgrading kitchen equipment, and paying daily expenses. The game begins with ₱500, 50 reputation points, and an initial supply of ingredients. Its objective is to complete seven days with at least ₱1,500 and 60 reputation points after paying the final day's expenses.

The program uses a numbered menu for keyboard input and displays information through text and ASCII artwork. Customers order cheeseburgers, chicken rice, French fries, or iced tea. The player must select the correct dish and cooking station while maintaining sufficient ingredients. Successful orders generate income and tips and improve reputation. Incorrect orders waste ingredients and reduce reputation.

The program demonstrates procedures, macros, input validation, and basic text-file handling within a single assembly source file. Procedures organize tasks such as serving customers, purchasing ingredients, and calculating daily expenses. The PRINT and NUMBER macros simplify text and numeric output. Invalid entries are rejected to prevent incorrect transactions.

For file handling, the player can append a game summary to reports.txt and read previously written reports through the menu. Each report records information such as cash, reputation, customer counts, kitchen level, and inventory. Reports remain available after the program closes, but they do not restore gameplay progress.

COMPILATION PROCESS:

```
● @kizern ➜/workspaces/bananabread/FINALPROJ (main) $ nasm -f elf32 food.asm -o food.o
● @kizern ➜/workspaces/bananabread/FINALPROJ (main) $ ld -m elf_i386 food.o -o food
@kizern ➜/workspaces/bananabread/FINALPROJ (main) $ ./food
```

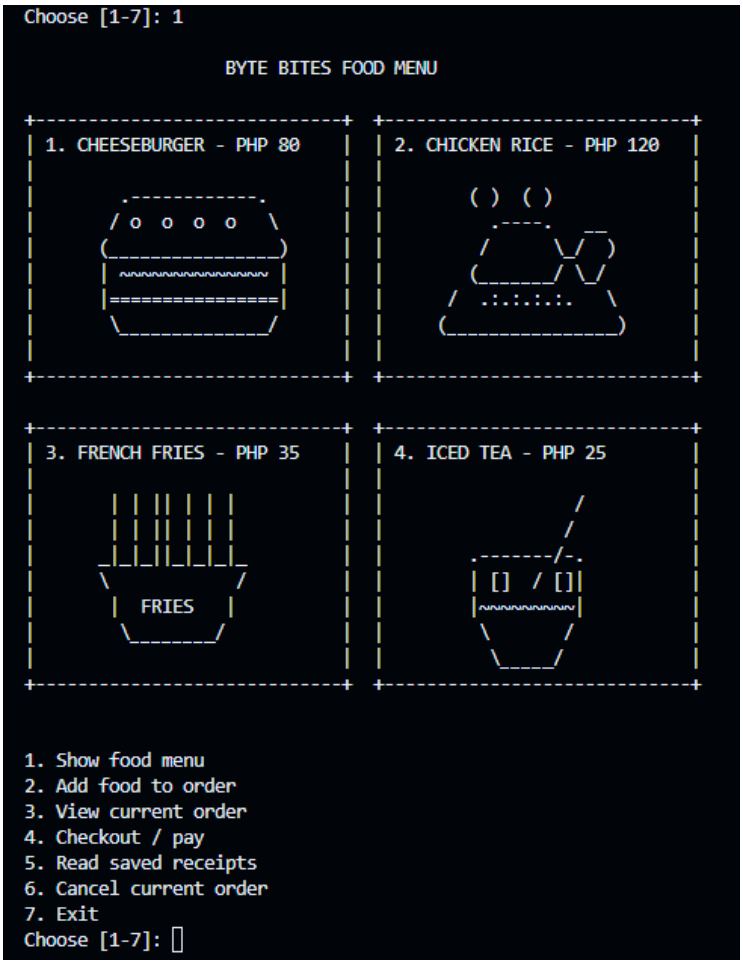
PROGRAM OUTPUT:

MAIN MENU:

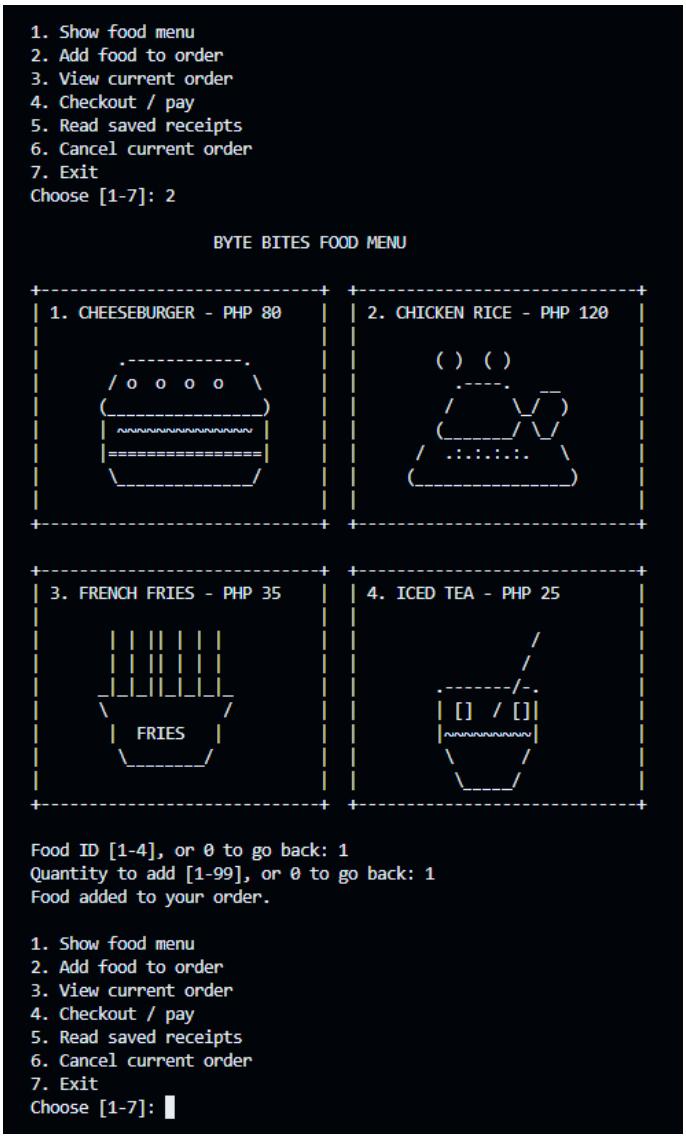
```
+=========================================+
|             BYTE BITES                  |
|          .---------------.              |
|         /  o   o   o   o  \             |
|        |====================|           |
|        | lettuce + burger   |           |
|         \__________________/             |
|          GOOD FOOD. GOOD MOOD.          |
+=========================================+

1. Show food menu
2. Add food to order
3. View current order
4. Checkout / pay
5. Read saved receipts
6. Cancel current order
7. Exit
Choose [1-7]:
```

OPTION 1:


Choose [1-7]: 1
BYTE BITES FOOD MENU
1. CHEESEBURGER - PHP 80
2. CHICKEN RICE - PHP 120
3. FRENCH FRIES - PHP 35
FRIES
4. ICED TEA - PHP 25
1. Show food menu
2. Add food to order
3. View current order
4. Checkout / pay
5. Read saved receipts
6. Cancel current order
7. Exit
Choose [1-7]:

OPTION 2:


1. Show food menu
2. Add food to order
3. View current order
4. Checkout / pay
5. Read saved receipts
6. Cancel current order
7. Exit
Choose [1-7]: 2
BYTE BITES FOOD MENU
1. CHEESEBURGER - PHP 80
2. CHICKEN RICE - PHP 120
3. FRENCH FRIES - PHP 35
FRIES
4. ICED TEA - PHP 25
Food ID [1-4], or 0 to go back: 1
Quantity to add [1-99], or 0 to go back: 1
Food added to your order.
1. Show food menu
2. Add food to order
3. View current order
4. Checkout / pay
5. Read saved receipts
6. Cancel current order
7. Exit
Choose [1-7]:

OPTION 3:

```
1. Show food menu
2. Add food to order
3. View current order
4. Checkout / pay
5. Read saved receipts
6. Cancel current order
7. Exit
Choose [1-7]: 3

--- CURRENT ORDER ---
Cheeseburger x 1 = PHP 80
TOTAL: PHP 80

1. Show food menu
2. Add food to order
3. View current order
4. Checkout / pay
5. Read saved receipts
6. Cancel current order
7. Exit
Choose [1-7]:
```

OPTION 4:

```
1. Show food menu
2. Add food to order
3. View current order
4. Checkout / pay
5. Read saved receipts
6. Cancel current order
7. Exit
Choose [1-7]: 4

--- CURRENT ORDER ---
Cheeseburger x 1 = PHP 80
TOTAL: PHP 80
Cash in whole PHP [1-1000000], or 0 to go back: 80
Confirm payment? 1 = pay, 0 = go back: 1
Payment recorded. Enjoy your meal!

========== BYTE BITES RECEIPT ==========
Cheeseburger x 1 = PHP 80
TOTAL: PHP 80
CASH: PHP 80
CHANGE: PHP 0
========================================

1. Show food menu
2. Add food to order
3. View current order
4. Checkout / pay
5. Read saved receipts
6. Cancel current order
7. Exit
Choose [1-7]:
```

OPTION 5:

```
========== BYTE BITES RECEIPT ==========
Cheeseburger x 1 = PHP 80
TOTAL: PHP 80
CASH: PHP 80
CHANGE: PHP 0
========================================

1. Show food menu
2. Add food to order
3. View current order
4. Checkout / pay
5. Read saved receipts
6. Cancel current order
7. Exit
Choose [1-7]:
```

OPTION 6:

```
1. Show food menu
2. Add food to order
3. View current order
4. Checkout / pay
5. Read saved receipts
6. Cancel current order
7. Exit
Choose [1-7]: 6
Order cancelled. Nothing saved.

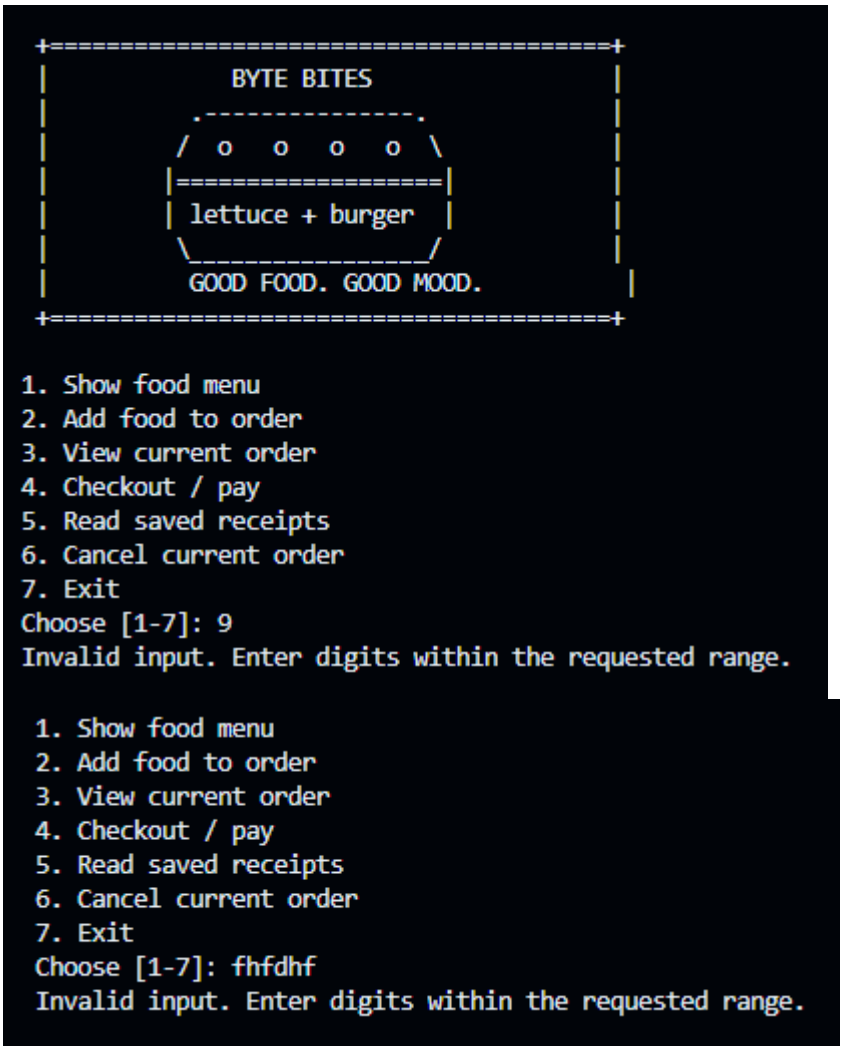
1. Show food menu
2. Add food to order
3. View current order
4. Checkout / pay
5. Read saved receipts
6. Cancel current order
7. Exit
Choose [1-7]: 3
Your order is empty. Add food first.

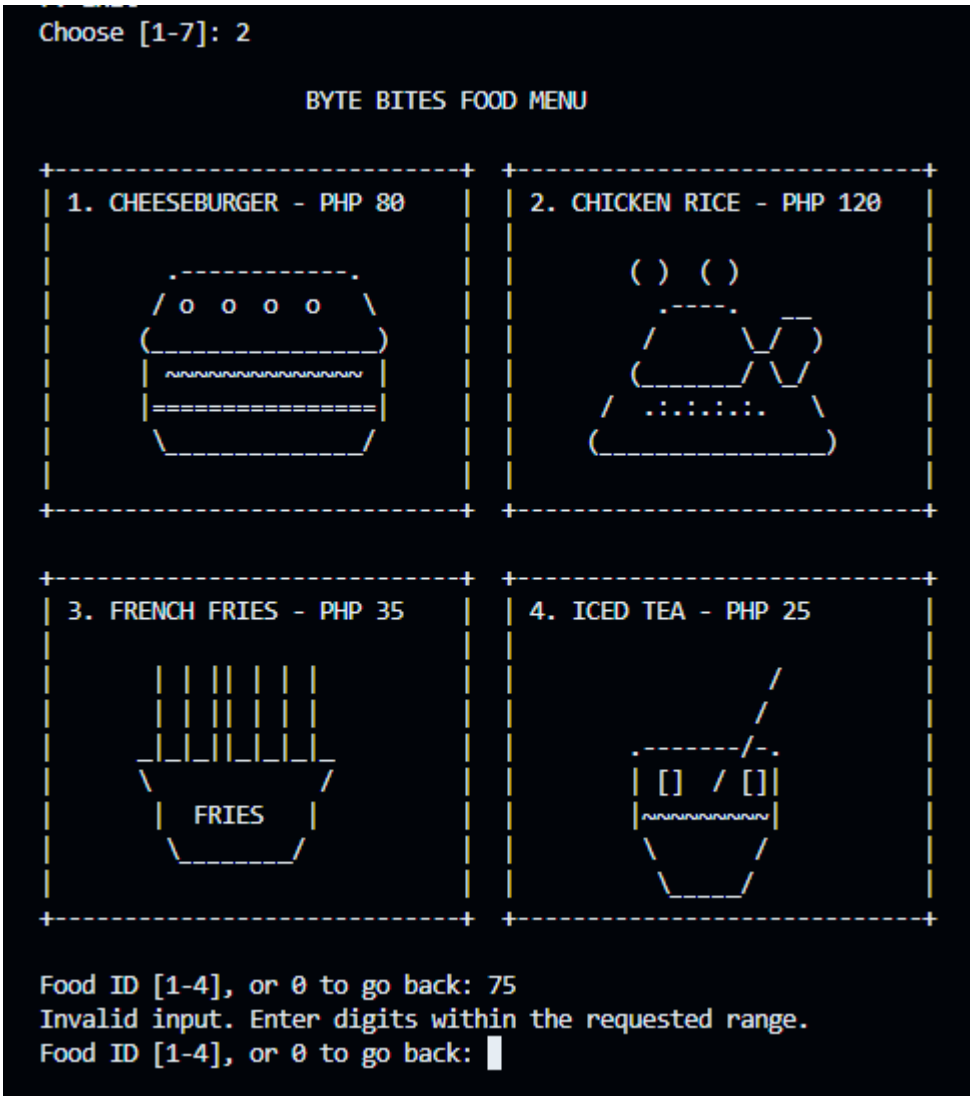
1. Show food menu
2. Add food to order
3. View current order
4. Checkout / pay
5. Read saved receipts
6. Cancel current order
7. Exit
Choose [1-7]:
```

OPTION 7:

```
1. Show food menu
2. Add food to order
3. View current order
4. Checkout / pay
5. Read saved receipts
6. Cancel current order
7. Exit
Choose [1-7]: 7
Thank you for visiting Byte Bites!
```

ERROR HANDLING:


BYTE BITES
lettuce + burger
GOOD FOOD. GOOD MOOD.
1. Show food menu
2. Add food to order
3. View current order
4. Checkout / pay
5. Read saved receipts
6. Cancel current order
7. Exit
Choose [1-7]: 9
Invalid input. Enter digits within the requested range.
1. Show food menu
2. Add food to order
3. View current order
4. Checkout / pay
5. Read saved receipts
6. Cancel current order
7. Exit
Choose [1-7]: fhfdhf
Invalid input. Enter digits within the requested range.



SOURCE CODE:

global _start

%macro PRINT 1 push ecx mov ecx, %1 call print_z pop ecx %endmacro

section .data banner db 10,'
+===+',10,' | BYTE BITES |',10,' | .--
----------. |',10,' | / o o o o \ |',10,' | |==================| |',10,' | | lettuce + burger |
|',10,' | _/ |',10,' | GOOD FOOD. GOOD MOOD. |',10,'
+===+',10,0 menu db 10,'1. Show food menu',10,'2. Add food to order',10,'3. View current order',10,'4. Checkout /

pay',10,'5. Read saved receipts',10,'6. Cancel current order',10,'7. Exit',10,'Choose [1-7]: ',0 food_menu: db '',10 db ' BYTE BITES FOOD MENU',10 db '',10 db '+----------------------+ +----------------------------+',10 db '| 1. CHEESEBURGER - PHP 80 | | 2. CHICKEN RICE - PHP 120 |',10 db '| | | |',10 db '| .------------. | | ( ) ( ) |',10 db '| / o o o o \ | | .----. __ |',10 db '| () | | / _/) |',10 db '| | ~~~~~~~~~~~~~~~~ | | | (/ _/ |',10 db '| |================| | | /.:.:.:.:. \ |',10 db '| _/ | | (__) |',10 db '| | | |',10 db '+----------------------+ +----------------------------+',10 db '',10 db '+----------------------------+ +----------------------------+',10 db '| 3. FRENCH FRIES - PHP 35 | | 4. ICED TEA - PHP 25 |',10 db '| | | |',10 db '| | | || | | | | | /|',10 db '| | | || | | | | | /|',10 db '| |||_|||_ | | .--------/-. |',10 db '| \ / | | | [] / [] | |',10 db '| | FRIES | | | |~~~~~~~~~~| |',10 db '| _/ | | \ / |',10 db '| | | ____/ |',10 db '+----------------------------+ +----------------------------+',10 db '',10 db 0
item_prompt db 'Food ID [1-4], or 0 to go back: ',0 qty_prompt db 'Quantity to add [1-99], or 0 to go back: ',0 cash_prompt db 'Cash in whole PHP [1-1000000], or 0 to go back: ',0 confirm_prompt db 'Confirm payment? 1 = pay, 0 = go back: ',0 invalid db 'Invalid input. Enter digits within the requested range.',10,0 limit_msg db 'Maximum quantity is 99 per food item in one order.',10,0 added db 'Food added to your order.',10,0 empty_msg db 'Your order is empty. Add food first.',10,0 cancel_msg db 'Order cancelled. Nothing saved.',10,0 short_msg db 'Insufficient cash. Please try again.',10,0 back_msg db 'Payment cancelled. Your order is still available.',10,0 saved_msg db 'Payment recorded. Enjoy your meal!',10,0 save_error db 'Could not save receipt. Check receipts.txt before retrying:',10,'a partial or complete record may already exist. Order retained.',10,0 history_missing db 'No saved receipts yet.',10,0 history_error db 'Unable to open, read, or close receipts.txt.',10,0 bye db 'Thank you for visiting Byte Bites!',10,0 input_error db 'Input error. Exiting.',10,0 order_title db 10,'--- CURRENT ORDER ---',10,0 receipt_title db 10,'========== BYTE BITES RECEIPT ==========',10,0 receipt_end db '==',10,0 qty_label db ' x ',0 line_label db ' = PHP ',0 total_label db 'TOTAL: PHP ',0 cash_label db 'CASH: PHP ',0 change_label db 'CHANGE: PHP ',0 newline db 10,0 name1 db 'Cheeseburger',0 name2 db 'Chicken Rice',0 name3 db 'French Fries',0 name4 db 'Iced Tea',0 names dd name1,name2,name3,name4 prices dd 80,120,35,25 filename db 'receipts.txt',0

section .bss cart resd 4 total resd 1 selected resd 1 cash resd 1 change resd 1 file_fd resd 1 receipt_len resd 1 receipt resb 1024 read_buffer resb 512 input_char resb 1 number_buffer resb 11

section .text _start: cld PRINT banner main_loop: PRINT menu call read_number cmp eax, -2 je exit_ok cmp eax, 1 je show_food cmp eax, 2 je add_food cmp eax, 3 je view_order cmp eax, 4 je checkout cmp eax, 5 je history cmp eax, 6 je cancel_order cmp eax, 7 je exit_ok PRINT invalid jmp main_loop show_food: PRINT food_menu jmp main_loop

add_food: PRINT food_menu .ask_item: PRINT item_prompt call read_number cmp eax, -2 je exit_ok test eax, eax jz main_loop cmp eax, 1 jl .bad_item cmp eax, 4 jg .bad_item

dec eax mov [selected], eax jmp .ask_qty .bad_item: PRINT invalid jmp .ask_item .ask_qty: PRINT qty_prompt call read_number cmp eax, -2 je exit_ok test eax, eax jz main_loop cmp eax, 1 jl .bad_qty cmp eax, 99 jg .bad_qty mov ebx, [selected] mov edx, [cart+ebx*4] add edx, eax cmp edx, 99 ja .limit mov [cart+ebx*4], edx imul eax, [prices+ebx*4] add [total], eax PRINT added jmp main_loop .bad_qty: PRINT invalid jmp .ask_qty .limit: PRINT limit_msg jmp .ask_qty

view_order: cmp dword [total], 0 je empty_order PRINT order_title call build_items mov byte [edi], 0 PRINT receipt jmp main_loop empty_order: PRINT empty_msg jmp main_loop cancel_order: call clear_cart PRINT cancel_msg jmp main_loop

checkout: cmp dword [total], 0 je empty_order PRINT order_title call build_items mov byte [edi], 0 PRINT receipt .ask_cash: PRINT cash_prompt call read_number cmp eax, -2 je exit_ok test eax, eax jz .back js .bad_cash cmp eax, [total] jb .short mov [cash], eax sub eax, [total] mov [change], eax .confirm: PRINT confirm_prompt call read_number cmp eax, -2 je exit_ok cmp eax, 0 je .back cmp eax, 1 je .save PRINT invalid jmp .confirm .bad_cash: PRINT invalid jmp .ask_cash .short: PRINT short_msg jmp .ask_cash .back: PRINT back_msg jmp main_loop .save: call build_receipt call save_receipt test eax, eax jnz .failed PRINT saved_msg PRINT receipt call clear_cart jmp main_loop .failed: PRINT save_error jmp main_loop

history: call read_history jmp main_loop exit_ok: PRINT bye mov eax, 1 xor ebx, ebx int 0x80

clear_cart: xor eax, eax mov edi, cart mov ecx, 4 rep stosd mov [total], eax mov [cash], eax mov [change], eax ret

read_number: push ebx push ecx push edx push esi push edi push ebp xor esi, esi xor edi, edi xor ebp, ebp .next: mov eax, 3 xor ebx, ebx mov ecx, input_char mov edx, 1 int 0x80 cmp eax, -4 je .next test eax, eax js .io_error jz .eof movzx eax, byte [input_char] cmp eax, 10 je .finish cmp eax, '0' jb .bad cmp eax, '9' ja .bad test ebp, ebp jnz .next inc edi cmp edi, 7 ja .bad sub eax, '0' imul esi, esi, 10 add esi, eax cmp esi, 1000000 ja .bad jmp .next .bad: mov ebp, 1 jmp .next .eof: mov eax, -2 test edi, edi jnz .finish test ebp, ebp jnz .finish jmp .done .finish: mov eax, -1 test ebp, ebp jnz .done test edi, edi jz .done mov eax, esi .done: pop ebp pop edi pop esi pop edx pop ecx pop ebx ret .io_error: PRINT input_error mov eax, 1 mov ebx, 1 int 0x80

write_all: push ecx push edx .next: test edx, edx jz .success mov eax, 4 int 0x80 cmp eax, -4 je .next test eax, eax js .done jz .zero_write add ecx, eax sub edx, eax jmp .next .zero_write: mov eax, -5 jmp .done .success: xor eax, eax .done: pop edx pop ecx ret

print_z: pushad xor edx, edx .count: cmp byte [ecx+edx], 0 je .write inc edx jmp .count .write: mov ebx, 1 call write_all test eax, eax js output_error popad ret output_error: mov eax, 1 mov ebx, 1 int 0x80

append_z: push eax push esi .next: lodsb test al, al jz .done stosb jmp .next .done: pop esi pop eax ret

append_uint: push eax push ebx push ecx push edx push esi mov esi, number_buffer+10 mov byte [esi], 0 mov ebx, 10 .convert: xor edx, edx div ebx add dl, '0' dec esi mov [esi], dl test eax, eax jnz .convert call append_z pop esi pop edx pop ecx pop ebx pop eax ret

append_items: xor ebp, ebp .next: mov eax, [cart+ebp*4] test eax, eax jz .skip mov esi, [names+ebp*4] call append_z mov esi, qty_label call append_z call append_uint mov esi, line_label call append_z imul eax, [prices+ebp*4] call append_uint mov esi, newline call append_z .skip: inc ebp cmp ebp, 4 jb .next mov esi, total_label call append_z mov eax, [total] call append_uint mov esi, newline call append_z ret

build_items: mov edi, receipt call append_items ret

build_receipt: mov edi, receipt mov esi, receipt_title call append_z call append_items mov esi, cash_label call append_z mov eax, [cash] call append_uint mov esi, newline call append_z mov esi, change_label call append_z mov eax, [change] call append_uint mov esi, newline call append_z mov esi, receipt_end call append_z mov byte [edi], 0 sub edi, receipt mov [receipt_len], edi ret

save_receipt: mov eax, 5 mov ebx, filename mov ecx, 1089 mov edx, 420 int 0x80 test eax, eax js .failed mov [file_fd], eax mov ebx, eax mov ecx, receipt mov edx, [receipt_len] call write_all test eax, eax js .close_failed .sync: mov eax, 118 mov ebx, [file_fd] int 0x80 cmp eax, -4 je .sync test eax, eax js .close_failed mov eax, 6 mov ebx, [file_fd] int 0x80 test eax, eax js .failed xor eax, eax ret .close_failed: mov eax, 6 mov ebx, [file_fd] int 0x80 .failed: mov eax, -1 ret

read_history: mov eax, 5 mov ebx, filename xor ecx, ecx int 0x80 cmp eax, -2 je .missing test eax, eax js .error mov [file_fd], eax .next: mov eax, 3 mov ebx, [file_fd] mov ecx, read_buffer mov edx, 512 int 0x80 cmp eax, -4 je .next test eax, eax js .close_error jz .close mov edx, eax mov ecx, read_buffer mov ebx, 1 call write_all test eax, eax js .close_error jmp .next .close: mov eax, 6 mov ebx, [file_fd] int 0x80 test eax, eax js .error ret .close_error: mov eax, 6 mov ebx, [file_fd] int 0x80 .error: PRINT history_error ret .missing: PRINT history_missing ret